How should web applications respond to scaling pressure?

Lecture 8 · Compare

Implement the smallest response that can be justified and remeasured.

Start from the observed limit

A credible scaling response connects:

1. the pressure that changed;
2. the boundary that saturated or failed;
3. the behavior that no longer met its goal;
4. the mechanism expected to change that relationship;
5. the cost and new obligation introduced.

Technique-first architecture produces complexity without a falsifiable reason.

Different bottlenecks justify different mechanisms

Observed boundary	Candidate response
Repeated expensive reads	Index, cache, or derive
One authoritative write path	Partition, serialize, batch, or reduce work
Slow non-critical work	Queue and process asynchronously
Dependency overload	Limit, isolate, shed, or degrade
Read locality or availability	Replicate with an explicit consistency model
Unequal tenants or consumers	Quotas, admission control, and isolation
Geographical latency	Move suitable reads or computation closer

Choose the smallest justified response

Prefer a bounded change such as:

- one missing index;
- one cache with an invalidation rule;
- one idempotency boundary;
- one admission or rate policy;
- one queued effect;
- one isolation boundary;
- one graceful-degradation path.

Preserve the accepted baseline so the response can be compared against it.

Remeasure under the unchanged question

Before and after the response, keep stable:

- acceptance behavior;
- workload and dataset;
- environment and revision identity;
- warm-up and repetition rules;
- collected distributions, errors, and resource signals.

Report all repetitions and negative results. A response that does not help is still useful evidence.

Apply formal models at two levels

Application decision

Use Little’s Law or another suitable relationship to check a queue, pool, request path, or conserved-work boundary.

Development-workflow decision

Use Amdahl’s Law to bound the value of parallel agent work when review or integration remains sequential.

Models discipline the argument; they do not replace measurement.

Every response creates a new operational obligation

Examples:

- a cache needs invalidation and cold-start behavior;
- a queue needs retry, deduplication, visibility, and backlog recovery;
- replication needs freshness and conflict rules;
- partitioning needs routing and rebalancing;
- rate limiting needs identity, fairness, and useful rejection behavior.

Exercise one failure and recovery path after the change.

Compare checkpoint

Submit:

- the A-B-C primary-pressure comparison;
- one implemented before-and-after response;
- its formal-model calculation;
- the new failure and recovery obligation;
- deployed workload evidence;
- a target architecture and rejected alternatives;
- the application and workflow trade-off table.

Question to carry forward: What can the three builds support as an engineering judgment?